Name: P . Lakshmi Gayathri Navaneetha
Reg No: 192321169

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 5 – CO5: Agents, Intelligent Systems and Real-Time AI

stacle avoidance because it must react within one tick. A learning component (estimating wind or battery drain) is optional; Task B1.3 adds a very small one, an adaptive battery-drain estimate.

A1.2 – Percepts, internal state and actions (1 mark)

Category	Content
Percepts (every tick)	gps = (x, y); battery (units); obstacle_dirs = subset of {N, E, S, W} whose neighbour cell is blocked; goal_blocked flag; clock t. The no-fly map is static knowledge.
Internal state remembered between steps	mode in {MISSION, RETURN, LANDED}; blocked = set of obstacle cells learned so far (map); delivered flag; home cell and goal cell; observed drain rate (battery units per move, starts at 1); previous battery value.
Actions	N, E, S, W (cost = drain rate), DROP (only on goal cell), LAND (at home, or emergency anywhere safe), HOVER (not used in the grid model).

A1.3 – Decision rule and priorities (1 mark)

Conflicts are resolved by a fixed priority order (lexicographic): **P1 safety > P2 battery reserve > P3 mission > P4 economy.**

- **P1 (L0 reactive safety).** Never enter a blocked, no-fly or out-of-bounds cell. A newly sensed obstacle in the intended direction triggers an immediate sidestep (N or S for an E/W move, E or W for N/S, first free one in that order).
- **P2 (L1 battery guard).** margin = battery − rate × dist(pos, home). If margin ≤ RESERVE (3) in MISSION mode, switch to RETURN. If battery ≤ 1 away from home, LAND immediately (emergency).
- **P3 (L2 deliberative).** Plan the shortest path to the goal (MISSION) or home (RETURN) on the known map.
- **P4 (economy).** Only among equal-length plans: prefer the move that shortens the larger coordinate gap first (x before y).

Utility. Hard rules P1 and P2 are constraints, not trade-offs. Among the plans that satisfy them, the planner maximises U = 100·delivered − 1·energy − 0.5·time − 20·risk, where risk counts cells next to buildings or no-fly zones. A delivery is worth 100 energy units, so a detour of up to about 100 moves is acceptable, but never at the cost of the reserve (P2).

A1.4 – Complete pseudocode (3 marks)

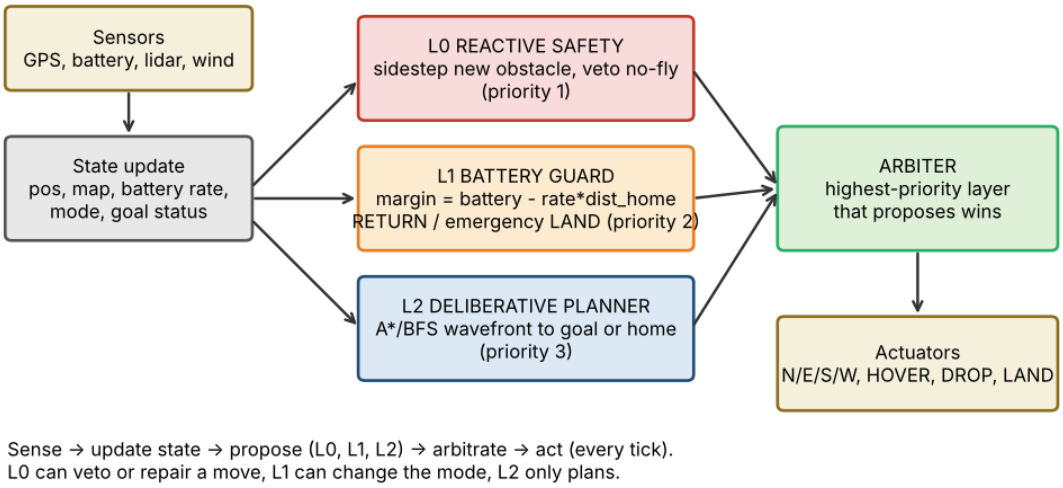


Figure A1 – Hybrid (layered) architecture with arbitration.

```
ALGORITHM DroneAgent(world)
  CONST RESERVE = 3
  STATE mode <- MISSION; blocked <- {}; rate <- 1; prev_battery <- NIL;
delivered <- FALSE
  home <- world.start; goal <- world.goal

  LOOP                                                  // sense - decide - act
cycle, once per tick
    p <- SENSE(world)                                   // gps, battery,
obstacle_dirs, goal_blocked

    // ---------- state update ----------
    FOR each d IN p.obstacle_dirs DO blocked <- blocked UNION {cell(p.gps, d)}
END FOR
    IF prev_battery != NIL AND prev_battery > p.battery THEN rate <- MAX(rate,
prev_battery - p.battery)
    prev_battery <- p.battery
    pos <- p.gps                                        // GPS re-estimate corrects
wind drift

    // ---------- proposals of the layers ----------
    target <- goal IF mode = MISSION ELSE home
    margin <- p.battery - rate * MANHATTAN(pos, home)

    // L1 battery guard (changes mode, or forces landing)
    IF mode = MISSION AND margin <= RESERVE THEN mode <- RETURN; target <- home
    IF mode = MISSION AND p.goal_blocked      THEN mode <- RETURN; target <-
home
    IF p.battery <= 1 AND pos != home         THEN act <- LAND; layer <- L1;
GOTO ACT
```

```
    // L2 deliberative planner
    IF mode = MISSION AND pos = goal THEN delivered <- TRUE; mode <- RETURN; act
<- DROP; GOTO ACT
    IF mode = RETURN  AND pos = home THEN mode <- LANDED;  act <- LAND; GOTO ACT
    dist_field <- BFS_FROM(target, avoiding blocked UNION nofly UNION
out_of_bounds)   // = A* with unit costs
    IF pos NOT IN dist_field THEN                    // target unreachable on
the known map
        IF mode = MISSION THEN mode <- RETURN; target <- home; recompute
dist_field
        IF pos NOT IN dist_field THEN act <- LAND; GOTO ACT
    act_plan <- first direction in [E, W, N, S] whose neighbour has dist_field =
dist_field[pos] - 1

    // L0 reactive safety (highest priority): was the planned move blocked only
because of NEW information?
    act_before <- plan computed WITHOUT the obstacles just sensed
    IF act_before IN p.obstacle_dirs THEN
        FOR each s IN lateral_directions(act_before) in order [N,S] or [E,W] DO
            IF cell(pos, s) is free THEN act <- s; layer <- L0; GOTO ACT
        act <- LAND; layer <- L0; GOTO ACT             // boxed in
    END IF
    act <- act_plan; layer <- L2

    // ---------- arbitration is the order above: L0 overrides L1 overrides L2 -
---------
    ACT: EXECUTE(act)                                // costs rate units for
moves
    IF act = LAND THEN RETURN (terminate)            // termination: landed (at
home or emergency)
    IF tick > MAX_TICKS THEN RETURN (terminate)      // watchdog
  END LOOP
```

A1.5 – Hand trace (1 mark)

Scenario: 8×8 grid, home H = (0,0), goal G = (6,0), battery 14 units, 1 unit per move, RESERVE = 3. An obstacle suddenly appears at (3,0) at t = 2. (x, y): x increases east, y increases north; the cell south of y = 0 does not exist.

t	Percept	Internal state	Layer that acted	Action
0	(0,0) b=14, none	MISSION; blocked={}; margin 14	L2	E
1	(1,0) b=13, none	MISSION; margin 12	L2	E
2	(2,0) b=12, OBSTACLE at E (3,0)	MISSION; blocked={(3,0)}; margin 10	L0 (new obstacle ahead)	N (S is outside the grid)
3	(2,1) b=11, none	MISSION; margin 8; new path via row 1	L2	E
4	(3,1) b=10, none	MISSION; margin 6	L2	E
5	(4,1) b=9, none	MISSION; margin 4 > 3	L2	E
6	(5,1) b=8, none	margin 2 ≤ 3, so mode becomes RETURN	L1 (battery overrides mission)	W
7	(4,1) b=7	RETURN; margin 2	L2	W

8	(3,1) b=6	RETURN; margin 2	L2	W
9	(2,1) b=5	RETURN	L2	W
10	(1,1) b=4	RETURN	L2	W
11	(0,1) b=3	RETURN	L2	S
12	(0,0) b=2	RETURN; at home so LANDED	L1	LAND

Outcome: the parcel is not delivered (goal 2 cells away at t = 6 but the reserve rule forbids it: reaching the goal and flying home would need 2 + 6 = 8 units while only 8 − 3 = 5 are usable). The drone lands at home with 2 units. Safety over mission over economy is demonstrated: L0 acts at t = 2 and L1 at t = 6.

TASK A2 – Don't Centralise What Can Be Local: 50-Drone Flood Search (5 marks)

A2.1 – Weakness of the centralised design (1 mark)

Failure	Effect on coverage	Effect on communication	Effect on mission success
Controller crashes	No new cells are assigned; every drone finishes its current cell and idles. Coverage freezes at whatever was done.	Nothing to talk to; 50 uplinks per tick are wasted.	Single point of failure; mission ends.
Links degrade	A drone with a lost command repeats or idles; cells are reassigned late.	Retransmissions increase the load at the controller (O(N) messages per tick into one node).	Slower coverage; time to 90% grows.
A drone dies	Its assigned cell stays 'taken' until the controller times out.	Controller must detect silence.	Small loss, but only if the controller is alive.

Measured evidence (Task B1, N = 10, 30 seeds). With perfect links the central design finds all survivors in 60% of runs; if the controller dies at tick 60 this drops to **23%** (coverage 84%), while the swarm has no such single point. The controller receives up to 10 messages per tick (grows linearly with N, 50 for 50 drones); a swarm node receives only messages from neighbours within its radio range.

A2.2 – Decentralised swarm strategy (2 marks)

Strategy: frontier selection with a local claim test (a Voronoi-like split computed independently by each drone). Each drone keeps its own coverage map, merges maps with neighbours inside its radio radius, and chooses the nearest uncovered cell that it is closer to than every neighbour it can hear. No drone assigns anything to another.

Information needed from neighbours (within radio radius RC): (1) their current positions, (2) their coverage maps (or only the newly covered cells since the last message). Nothing else: no global plan, no identity table.

```
RULE SwarmStep(i)                                    // runs on every drone i,
once per tick
  neighbours <- { j : MANHATTAN(pos[i], pos[j]) <= RC }          // from radio
messages
  known[i]   <- known[i] UNION ( UNION of known[j] for j IN neighbours )      //
map merge
  IF battery[i] - dist(pos[i], home[i]) <= RESERVE THEN target <- home[i]    //
L1 battery guard first
  ELSE
     U <- { cells c : c is reachable AND c NOT IN known[i] }
     IF U is empty THEN target <- home[i]
     ELSE
```

```
        claimed <- { c IN U : FOR ALL j IN neighbours:
                        dist(pos[i],c) < dist(pos[j],c)  OR  (dist(pos[i],c) =
dist(pos[j],c) AND i < j) }
        IF claimed is not empty THEN target <- argmin over claimed of
dist(pos[i], c)
        ELSE target <- argmin over U of dist(pos[i], c)           // fallback:
nothing is uniquely mine
  MOVE one step toward target (x or y first, sidestep building or occupied cell)
  MARK cells within sensing footprint as covered in known[i]; BROADCAST position
and new cells
```

A2.3 – Emergent coverage on a 5×5 grid, three drones (1 mark)

Assumptions for the hand trace: sensing covers only the drone's own cell, all three drones hear each other, drones move simultaneously, one step per tick, x-axis step first. Start: D1 (0,0), D2 (4,0), D3 (2,4); the three start cells are covered at t = 0 (22 cells unvisited).

t	Targets chosen (D1, D2, D3)	Positions after move	Newly covered	Unvisited left
1	(0,1), (3,0), (1,4)	(0,1), (3,0), (1,4)	3	19
2	(0,2), (2,0), (0,4)	(0,2), (2,0), (0,4)	3	16
3	(0,3), (1,0), (3,4)	(0,3), (1,0), (1,4)*	2	14
4	(1,3), (1,1), (3,4)	(1,3), (1,1), (2,4)	2	12
5	(1,2), (2,1), (3,4)	(1,2), (2,1), (3,4)	3	9
6	(2,2), (3,1), (3,3)	(2,2), (3,1), (3,3)	3	6
7	(2,3), (4,1), (4,3)	(2,3), (4,1), (4,3)	3	3
8	(3,2), (4,2), (4,4)	(3,3), (4,2), (4,4)	2	1
9	(3,2), (3,2), (3,2)	(3,2), (3,2), (3,4)	1	0

*At t = 3 D3 walks from (0,4) to (1,4), a cell it already covered at t = 1 (its target (3,4) lies beyond it), so that move is a repeated visit: a small redundancy caused by the walk to a far target. **Emergence:** D1 sweeps the left columns, D2 the bottom row and middle, D3 the top row and right side without any message saying so; the partition appears because each drone only claims cells it is closest to. All 25 cells are covered in 9 ticks; one drone alone needs at least 24 ticks. At t = 9 all three drones fall back to the same last cell (3,2), because only one cell is left and the claim test leaves no unique owner (2 wasted moves).

A2.4 – Challenge the rule: a failing configuration (1 mark)

Configuration. Radio radius RC = 2 (very limited). Everything is covered except (2,2) and (2,3). D1 at (0,2), D2 at (4,2), D3 at (0,0).

Prediction (made before tracing): D1 and D2 are 4 apart, so neither hears the other. Both believe they are the only drone near (2,2), both walk there and arrive together at t = 2, then both walk to (2,3) at t = 3. Two drones will finish no faster than one (3 ticks) and one visit per cell is redundant. D3 hears D1 (distance 2) and should stay away.

t	Targets (D1, D2, D3)	Positions after move	Newly covered	Unvisited left
1	(2,2), (2,2), (2,2)	(1,2), (3,2), (1,0)	0	2
2	(2,2), (2,2), (2,2)	(2,2), (2,2), (2,0)	1	1
3	(2,3), (2,3), (2,3)	(2,3), (2,3), (2,1)	1	0

Result. The prediction for D1 and D2 was right: they duplicate each other for two ticks (redundant coverage 100% on both cells) and coverage finishes at t = 3, exactly the time one drone alone would need. The prediction for D3 was wrong: the claim test leaves D3 with no cell it is nearer to than D1, so the **fallback line**

(take the nearest uncovered cell) sends it towards the same cells, so all three drones cluster on the last two cells. Lesson: limited communication range creates blind spots, and the fallback creates clustering at the end. Possible repair: in the fallback, pick the uncovered cell farthest from all heard neighbours, or return home when no unique cell exists.

TASK A3 – Discover the Expert-System Architecture Pattern (5 marks)

A3.1 – Common pattern (2 marks)

System	Rule set (IF … THEN …)
1. Pump-fault advisor	P1: IF noise=crackling AND suction_pressure=low THEN fault=cavitation P2: IF leak_visible=yes THEN fault=seal_leak P3: IF noise=grinding AND bearing_temp=high THEN fault=bearing_failure P4: IF flow=low AND motor_current=low AND suction_pressure=normal THEN fault=impeller_wear P5: IF motor_current=high AND motor_temp=high THEN fault=motor_overload
2. Crop-disease advisor	C1: IF leaf_spots=yellow_halo AND humidity=high THEN disease=bacterial_blight C2: IF leaf_coating=white_powder THEN disease=powdery_mildew C3: IF stem_lesions=dark AND weather=wet THEN disease=anthracnose C4: IF disease=powdery_mildew AND crop_stage=flowering THEN treatment=sulphur_spray C5: IF disease=bacterial_blight THEN treatment=copper_fungicide
3. Flood-relief triage	F1: IF people_trapped=yes AND water_level=rising THEN priority=critical F2: IF no_drinking_water=yes AND children_present=yes THEN priority=high F3: IF road_access=cut AND medical_need=yes THEN priority=critical F4: IF shelter_damaged=yes AND people_trapped=no THEN priority=medium F5: IF only_property_damage=yes THEN priority=low

Common to all three (reusable without code changes)	Domain-specific (knowledge, not code)
IF-conjunction-THEN rule format; facts as attribute=value pairs; forward chaining (match, select, fire, repeat); conflict resolution (specificity, priority, recency); chained inference (disease then treatment; sensor then derived fact); explanation of results; handling of missing facts by asking.	The attribute names and values (noise, leaf_spots, water_level …); the rule contents; the goal slot (fault / disease+treatment / priority); the question wording; priorities and certainty.

What can be reused unchanged: the inference engine, working memory, conflict-resolution strategies, explanation facility (HOW/WHY) and the rule editor. Only the rule file changes. This is exactly what the expert-system shell of A3.2 and the program of Task B2 implement.

A3.2 – Generic expert-system shell (2 marks)

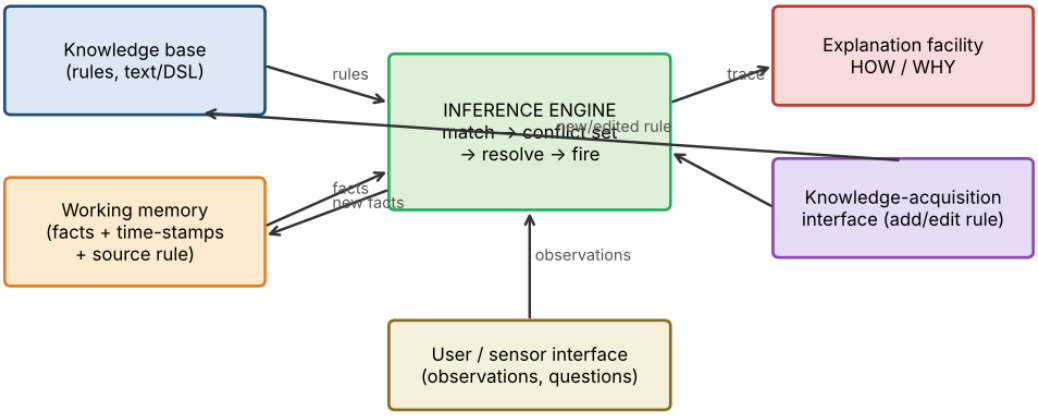


Figure A3 – Components of the generic shell and what flows between them.

Component	Holds / does	Data exchanged
Knowledge base	Rules (id, premises, conclusion, priority) stored as text; no domain code in the engine.	Rules → inference engine; new or edited rules ← knowledge-acquisition interface.
Working memory	Facts attribute=value with time-stamp and source (given / asked / rule id).	Facts ↔ inference engine; facts → explanation facility.
Inference engine	Match, conflict set, resolution, fire, repeat until no rule is applicable (refraction: each rule fires once).	Conflict set; fired-rule trace → explanation.
Explanation facility	HOW (derivation chain of a fact), WHY (why a question is asked).	Trace ← engine; text → user.
Knowledge-acquisition interface	Add / edit / delete rules at run time with syntax check; regression run.	Rule text → knowledge base; before/after diagnoses → expert.

```
ALGORITHM InferenceCycle(KB, observations, strategy)
  WM <- { (a = v, stamp 0, source 'given') FOR each observation }
  fired <- {}; clock <- 0
  LOOP
    conflict_set <- { r IN KB : r NOT IN fired AND ALL premises of r hold in WM
}       // MATCH
    IF conflict_set is empty THEN
        IF goal slot empty AND a question can complete exactly one missing
premise THEN
            ask user (explain WHY); add answer to WM; CONTINUE
        EXIT LOOP
    END IF
    r <- SELECT from conflict_set by strategy:
// CONFLICT RESOLUTION
           specificity : most premises first            (tie: rule order)
           priority    : highest priority number first  (tie: rule order)
           recency     : rule using the newest fact first (tie: rule order)
    clock <- clock + 1
    IF r concludes the goal slot AND the slot is already filled THEN record r as
'suppressed'
```

```
    ELSE add conclusion to WM with (stamp clock, source r.id)
// FIRE
    fired <- fired UNION {r}
  END LOOP
  RETURN goal slot value (or 'undetermined'), fired list, WM          // used by
HOW and WHY
```

A3.3 – Why explanation and knowledge acquisition are essential (1 mark)

Domain experts (pump engineers, agronomists, relief coordinators) are not programmers and are professionally accountable for the advice they act on. **HOW** ("fault = bearing_failure because R11: noise = grinding, vibration = high") lets them check the chain of reasoning against their own judgement, spot a wrong rule and decide whether to trust the conclusion; without it the system is a black box nobody can responsibly use. **WHY** ("I ask suction_pressure because rules R05–R07 need it") makes the dialogue understandable and keeps the number of questions small. The **knowledge-acquisition interface** lets the expert correct or extend rules directly when the system errs or the plant changes; without it every change needs a programmer, the knowledge becomes stale and the experts stop using the tool. Together they close the loop: explanation finds the faulty knowledge, acquisition repairs it, and a regression run shows what else the repair changed (Task B2.3).

TASK A4 – Distributed Data Pipeline with RDDs (4 marks)

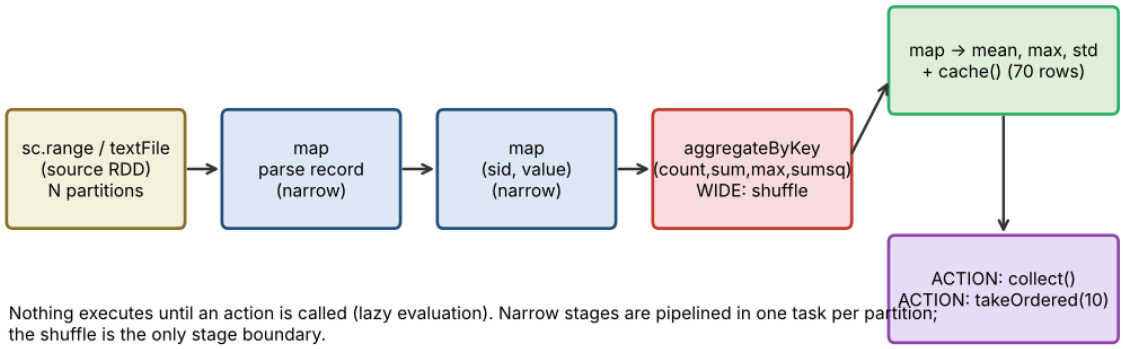


Figure A4 – RDD pipeline: narrow steps are pipelined, the aggregateByKey is the only shuffle.

```
// readings: (sensorId, timestamp, value) - about 10 million per day
raw     <- sc.textFile("readings/day.csv", minPartitions = 64)   // source RDD:
no work yet
parsed  <- raw.map(parse_line)                                    //
TRANSFORMATION, narrow
valid   <- parsed.filter(value is not missing)                    //
TRANSFORMATION, narrow
kv      <- valid.map( r -> (r.sensorId, r.value) )                //
TRANSFORMATION, narrow

// per-key (count, sum, max, sum of squares) with map-side combine, so little
data is shuffled
```

```
agg     <- kv.aggregateByKey( (0,0,-INF,0), seqOp, combOp )       //
TRANSFORMATION, WIDE: shuffle by key
stats   <- agg.map( (id,(n,s,mx,ss)) -> (id, (s/n, mx, SQRT(ss/n - (s/n)^2))) )
// narrow
stats.cache()                                                     // ~5,000
sensors: tiny, reused

result1 <- stats.collect()                                        // ACTION 1:
mean and max per sensor
top10   <- stats.takeOrdered(10, key = -std)                      // ACTION 2:
ten noisiest sensors
```

Operation	Type	Narrow / wide	Comment
textFile, parallelize / range	creates RDD	-	lazy; only records lineage
map(parse), filter, map(key,value)	transformation	narrow	each output partition depends on one input partition; the three steps are fused into one task per partition
aggregateByKey	transformation	WIDE (shuffle)	records with the same sensorId must meet; map-side combine reduces shuffled data to (partitions × sensors) tuples
map (mean, max, std)	transformation	narrow	operates on aggregated rows
cache	hint	-	marks stats for reuse
collect, takeOrdered	ACTION	-	only these trigger execution (lazy evaluation); each action submits one job

When evaluation happens. Nothing runs while the transformations are written; Spark only builds the lineage graph. The first action (collect) triggers a job: stage 1 pipelines parse, filter and key-value inside each partition and writes shuffle files; stage 2 reads them and finishes the aggregation.

Caching decision. Cache stats: it has about 5,000 rows, costs almost no memory and is used by two actions. Do not cache kv / the raw readings unless the data is read again: 10 million readings would waste memory. In my measurement, caching the raw (key, value) RDD of 10^7 records used **96 MB** of storage memory (10^6 records: 9.6 MB) – and, as B3.3 shows, it only pays off when a third pass over the raw data is needed. A useful side effect observed in B3.3: Spark keeps shuffle output files, so even without cache() the second action does not redo the shuffle (stage skipped), which is why caching stats saves little in practice.

Lineage and recovery. Every RDD stores how it was derived from its parent (textFile → map → filter → map → aggregateByKey → map). If an executor dies and partition k of stats is lost, Spark does not need a replica: it recomputes only that partition by re-reading the corresponding input blocks and replaying the narrow steps (and, for a wide dependency, re-fetching or regenerating the shuffle files it needs). Narrow dependencies recover cheaply (one parent partition); wide dependencies may need many parent partitions, which is why long lineages across shuffles are check-pointed in practice.

TASK A5 – Design When the Full Model Is Too Expensive (4 marks)

Chosen case: AI-based intrusion detection on a network gateway (the same case as Task B3). The device budget used throughout is **50 ms per sample**, an assumed edge CPU about **8× slower** than my test CPU, and an assumed **10 MB** model-memory budget (these two device numbers are assumptions; replace them with your real device).

A5.1 – Full-model perspective (1 mark)

Quantity	Full model (random forest, 200 trees, unlimited depth)	How obtained
Input features	122 (38 numeric + one-hot of protocol, service, flag)	after encoding

Parameters (tree nodes)	234,524 nodes; each stores feature id, threshold, two children and class counts, i.e. about 0.9 million stored numbers	counted from the trained model
Memory	18.8 MB serialised (more in RAM)	pickle size
Operations per sample	≈ 5,901 comparisons (200 trees × mean depth 29.5, max 39) + 200 votes + encoding of 122 features	calculated
Measured latency (this CPU)	9.7 ms mean, 14.3 ms p95	Task B3.4
Projected on the edge device	≈ 78 ms mean, 114 ms p95 (×8 assumption)	assumption

Why it exceeds the budget. On a device 8× slower the mean latency (≈78 ms) and especially the p95 (≈114 ms) pass the 50 ms limit, and the 18.8 MB model is almost twice the assumed 10 MB memory budget. Important honesty note: on my own CPU the full model does meet 50 ms; it only fails on the assumed edge device, so the argument holds only if the device really is that slow.

A5.2 – Compressed pipeline (2 marks)

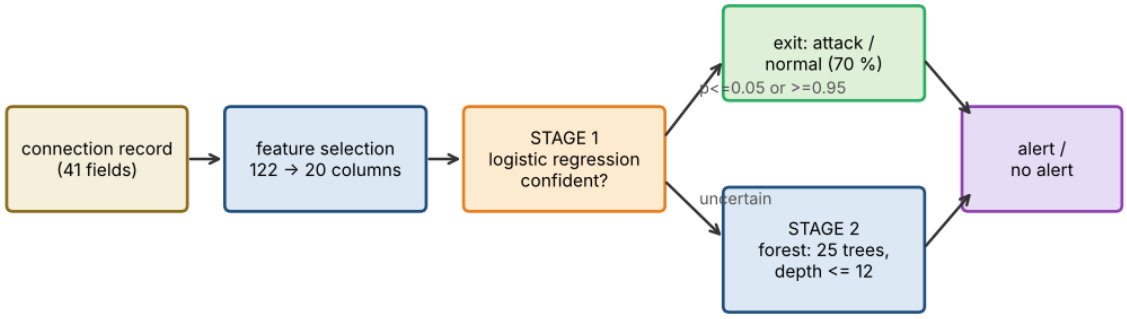


Figure A5 – Compressed pipeline: feature reduction, cheap first stage, small forest only when uncertain.

```
ALGORITHM EdgeDetect(record)
  x <- encode(record) restricted to the 20 most important columns      //
feature reduction 122 -> 20
  p1 <- LOGISTIC(x)                                                     // stage
1: 20 multiply-adds
  IF p1 <= 0.05 OR p1 >= 0.95 THEN RETURN (p1 >= 0.5)                   // early
exit
  p2 <- AVERAGE over 25 trees of depth <= 12 of TREE_VOTE(tree, x)      // stage
2: small forest
  RETURN (p2 >= 0.5)
```

Saved by	What is saved
Feature reduction (122 → 20 columns)	Encoding work and memory of 102 columns; every tree has fewer candidate features.
Smaller forest (200 → 25 trees, depth ≤ 12, min leaf 5)	Nodes 234,524 → 12,347 (5.3%); comparisons per sample ≈ 5,901 → at most 25 × 12 = 300.
Early-exit cascade	A logistic regression answers 70% of records without touching the forest.

A5.3 – What have we sacrificed? (1 mark)

Possible loss	Why	How I would detect it
Accuracy / recall	Fewer, shallower trees and fewer features give a coarser decision surface.	Compare accuracy, recall, F1, AUC against the full model on the same hold-out set (Task B3.4).
Robustness to unseen attacks	Reduced features may drop the rare signals that let the full model catch novel attack types.	Per-attack-type and seen/unseen recall (Task B3.2) for both models.
Calibration of confidence	The early-exit thresholds rely on the first stage being right when it is confident; confident but wrong answers skip the forest.	Error rate among early exits (measured in B3.4: see table), reliability curve.
Explainability	Cascades have two decision paths; fewer features change the explanation.	Compare the top features used for flagged records between models; keep the HOW/WHY text consistent.
Drift sensitivity	Small models are less able to absorb new traffic patterns.	Shadow-run the full model in the cloud on a sample and monitor disagreement.

PART B – IMPLEMENTATION, INSTRUMENTATION AND DEPLOYMENT DECISIONS (25 marks)

Source files (all in the zip): drone_agent.py, swarm.py, expert_shell.py, b2_pump.py, b2_run.py, intrusion.py, spark_scale.py, chatbot.py, figs.py, a2_trace.py. Random seeds are fixed: world seed = seed index, other streams derived from (seed, N, failure rate); numpy/sklearn seed 0; Python random seeds 7 and 42.

TASK B1 – Implement and Instrument the Drone Agent (7 marks)

B1.1 – Validate the implementation (2 marks)

Source code: drone_agent.py (class World = simulator, class DroneAgent/LayeredDroneAgent = agent logic, run_scenario = I/O and logging; the agent never reads the world's hidden state, only the percept).

Scenario (input)	What it verifies	Expected behaviour	Actual outcome
S1 normal: 8×8, goal (6,0), battery 30	Baseline plan, delivery, return, landing	Deliver, return, land	delivered=True, landed at home, battery left 18, 14 ticks
S2 sudden obstacle at (3,0) at t=2	L0 reactive sidestep and replanning	Sidestep, detour, still deliver	delivered=True, L0 fired at t=2 and again on the way back, battery left 14
S3a boundary: battery 2 at launch	Battery guard at takeoff	Never take off, land at home	landed at home immediately (1 tick), crashed=False
S3b boundary: drain triples after t=4 (headwind)	Guard when energy model is wrong	Return or land safely	delivered but emergency LAND at (2,0), 2 cells short of home, battery 0; no crash (guard assumed 1 unit/move)
S3b2 same, adaptive drain-rate estimate	Improved L1	Abort early and reach home	abort at t=5, lands at home, battery 0, delivered=False
S3c boundary: goal cell becomes blocked at t=4	Goal-blocked handling	Abort mission, return	RETURN at t=4 'goal blocked', landed at home, battery 22
S4 no-fly zone (3,0),(3,1)	Geofence respected	Route around zone, never enter	delivered=True, crashed=False, path detours via row 2
S5 wind drift (30% slip)	State update from GPS	Still delivers	delivered=True, landed at home

Behaviour log – S2 sudden obstacle:

t	gps	bat	sensed	mode	layer	action	reason
0	(0, 0)	30	-	MISSION	L2	E	
1	(1, 0)	29	-	MISSION	L2	E	
2	(2, 0)	28	E	MISSION	L0	N	new obstacle ahead (E) -> sidestep N
3	(2, 1)	27	-	MISSION	L2	E	
4	(3, 1)	26	S	MISSION	L2	E	
5	(4, 1)	25	-	MISSION	L2	E	
6	(5, 1)	24	-	MISSION	L2	E	
7	(6, 1)	23	-	MISSION	L2	S	
8	(6, 0)	22	-	RETURN	L2	DROP	parcel delivered
9	(6, 0)	22	-	RETURN	L2	W	
10	(5, 0)	21	-	RETURN	L2	W	
11	(4, 0)	20	W	RETURN	L0	N	new obstacle ahead (W) -> sidestep N
12	(4, 1)	19	-	RETURN	L2	W	
13	(3, 1)	18	S	RETURN	L2	W	
14	(2, 1)	17	-	RETURN	L2	W	
15	(1, 1)	16	-	RETURN	L2	W	
16	(0, 1)	15	-	RETURN	L2	S	
17	(0, 0)	14	-	LANDED	L1	LAND	at home, landing

Behaviour log – S3b headwind drain (original guard) and S3b2 (adaptive drain rate):

t	gps	bat	sensed	mode	layer	action	reason
0	(0, 0)	22	-	MISSION	L2	E	
1	(1, 0)	21	-	MISSION	L2	E	
2	(2, 0)	20	-	MISSION	L2	E	
3	(3, 0)	19	-	MISSION	L2	E	
4	(4, 0)	18	-	MISSION	L2	E	
5	(5, 0)	15	-	MISSION	L2	E	
6	(6, 0)	12	-	RETURN	L2	DROP	parcel delivered
7	(6, 0)	12	-	RETURN	L2	W	
8	(5, 0)	9	-	RETURN	L2	W	
9	(4, 0)	6	-	RETURN	L2	W	
10	(3, 0)	3	-	RETURN	L2	W	
11	(2, 0)	0	-	RETURN	L1	LAND	emergency landing (battery<=1)

t	gps	bat	sensed	mode	layer	action	reason
0	(0, 0)	22	-	MISSION	L2	E	
1	(1, 0)	21	-	MISSION	L2	E	
2	(2, 0)	20	-	MISSION	L2	E	
3	(3, 0)	19	-	MISSION	L2	E	
4	(4, 0)	18	-	MISSION	L2	E	
5	(5, 0)	15	-	RETURN	L1	W	battery margin 0<=RESERVE
6	(4, 0)	12	-	RETURN	L2	W	
7	(3, 0)	9	-	RETURN	L2	W	
8	(2, 0)	6	-	RETURN	L2	W	
9	(1, 0)	3	-	RETURN	L2	W	
10	(0, 0)	0	-	LANDED	L1	LAND	at home, landing

Boundary and failure tests: S3a, S3b, S3c above. **Interpretation:** in all eight scenarios the agent never crashed or entered a no-fly cell (safety invariant). S3b exposes a real weakness: the guard assumed 1 unit per move, so with a tripled drain the drone had to make an emergency landing 2 cells from home. Estimating the drain rate online (S3b2) fixes it at the price of abandoning a delivery that the optimistic guard would have completed, which is the right trade-off under P1/P2 > P3. Full logs for S3a, S3c, S4, S5 are in out_b1_scenarios.json.

B1.2 – Compare design with implementation (1 mark)

t	Hand-trace action / layer	Program action / layer	Same?
0	E / L2	E / L2	yes
1	E / L2	E / L2	yes
2	N / L0	N / L0	yes
3	E / L2	E / L2	yes
4	E / L2	E / L2	yes
5	E / L2	E / L2	yes
6	W / L1	W / L1	yes
7	W / L2	W / L2	yes
8	W / L2	W / L2	yes
9	W / L2	W / L2	yes
10	W / L2	W / L2	yes
11	S / L2	S / L2	yes
12	LAND / L1	LAND / L1	yes

Result: all 13 ticks agree on action, layer and battery. Differences found while comparing, and what they mean:

- **Bug found by the comparison (fixed).** In the first version the program labelled the t = 6 mode switch as layer L2 although my design says the battery guard (L1) acted. The decision was correct but the label was wrong, because the mode switch was coded inside the planner branch. I fixed the label; the logged layer is now L1 as in the hand trace.
- **Percept detail.** At t = 4 and t = 8 the program's percept lists obstacle 'S' (the known obstacle (3,0) lies below (3,1)). My hand trace wrote 'none'. It has no effect on the action because the obstacle is already in the map and the planner routes around it, but the hand trace should list adjacent known obstacles too.
- **Tie-break.** The hand trace assumed 'x-first' for equal shortest paths; the program implements this with the order [E, W, N, S] on the distance field. No difference appeared in this scenario, but only because the shortest paths here were unique after the first step.
- **Planner.** The hand design says A*; the code uses a BFS wavefront. On a unit-cost grid they produce the same shortest-path lengths, so this is an implementation simplification, not a behavioural difference.

B1.3 – Improve the design: swarm coordination (2 marks)

Improvement: N drones run the decentralised rule of Task A2.2 (frontier selection with local claim test, map merge within radio radius RC = 8) on top of the same L1 battery guard and a reactive sidestep around buildings and occupied cells (swarm.py, functions swarm_decide and run_mission). **What it targets:** (1) the single-drone limit (battery 120 moves covers only about 20% of the 30×30 city with 8% buildings), (2) the single point of failure of central control, (3) redundant coverage. A central-controller version of the same simulator is included only as the baseline (A2.1).

B1.4 – Measure the effect (2 marks)

Setup: 30×30 grid, 8% buildings, 10 hidden survivors, deadline 300 ticks, battery 120 moves, sensing radius 1 (5 cells), RC = 8, 30 seeds per cell of the table, N ∈ {1, 5, 10, 25, 50}, failure rate ∈ {0, 10, 30}% of drones (failures at a random tick in [20, 200); for N = 1 failure is a Bernoulli event). Mission success = all survivors located by the deadline. Redundancy ratio = 1 − (unique cells covered) / (cell-sensing events); even a perfect sweep has about 0.6 because each move re-senses 2 of 5 cells. Decision latency is the mean wall-clock time of one drone decision on this CPU.

N	fail	success	final cov. (± sd)	t to 90% (ticks)	redundancy	battery/drone	battery total	latency µs	drones out of battery
1	0%	0%	20.5% ± 4.3	never	0.67	108	108	19	0.0

1	10%	0%	21.0% ± 2.8	never	0.66	111	111	23	0.0
1	30%	0%	19.0% ± 5.0	never	0.66	99	99	20	0.0
5	0%	7%	74.3% ± 5.3	never	0.78	111	554	27	0.6
5	10%	3%	72.4% ± 5.2	never	0.78	107	535	25	0.5
5	30%	7%	71.1% ± 5.9	never	0.77	106	528	32	0.3
10	0%	57%	93.3% ± 4.6	75 (80% runs)	0.87	112	1121	43	1.7
10	10%	63%	93.5% ± 5.5	73 (70% runs)	0.85	106	1059	37	1.3
10	30%	50%	92.2% ± 4.5	77 (73% runs)	0.85	102	1019	35	1.0
25	0%	100%	100.0% ± 0.0	49 (100% runs)	0.88	56	1399	90	0.0
25	10%	100%	100.0% ± 0.0	48 (100% runs)	0.88	57	1428	89	0.0
25	30%	100%	100.0% ± 0.0	48 (100% runs)	0.87	54	1353	83	0.0
50	0%	100%	100.0% ± 0.0	40 (100% runs)	0.93	45	2266	134	0.0
50	10%	100%	100.0% ± 0.0	40 (100% runs)	0.93	45	2274	135	0.0
50	30%	100%	100.0% ± 0.0	40 (100% runs)	0.93	45	2240	131	0.0

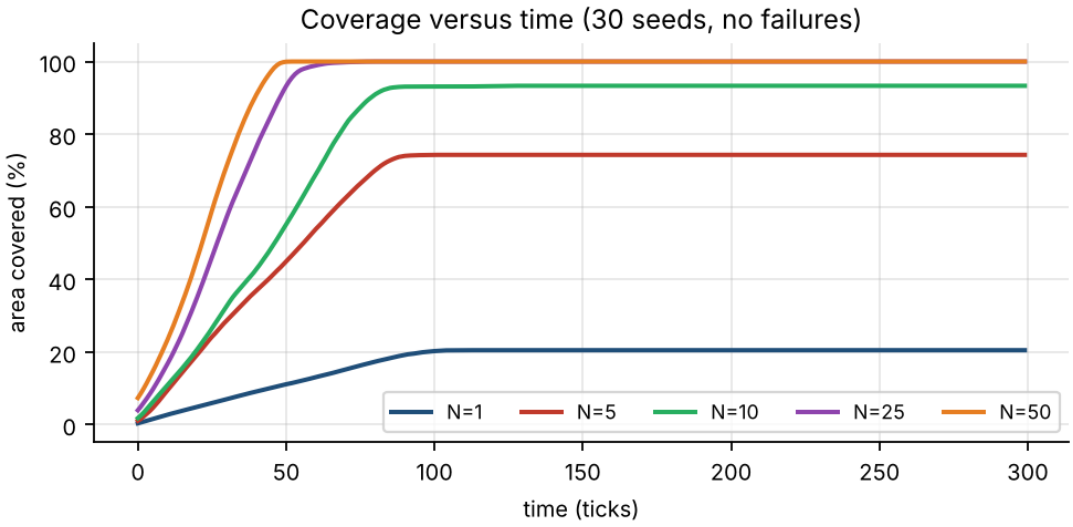


Figure B1a – Coverage against time.

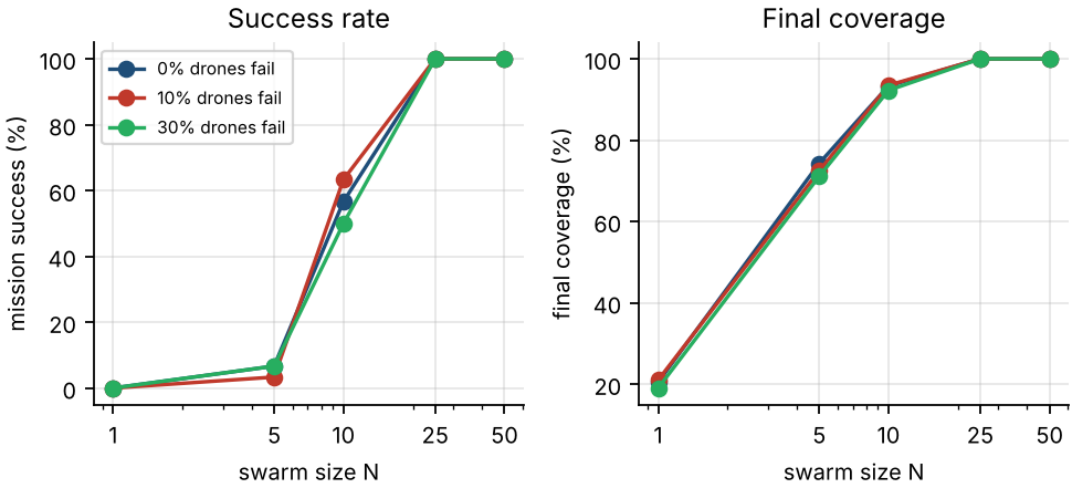


Figure B1b – Success and final coverage against N for 0, 10 and 30% drone failure.

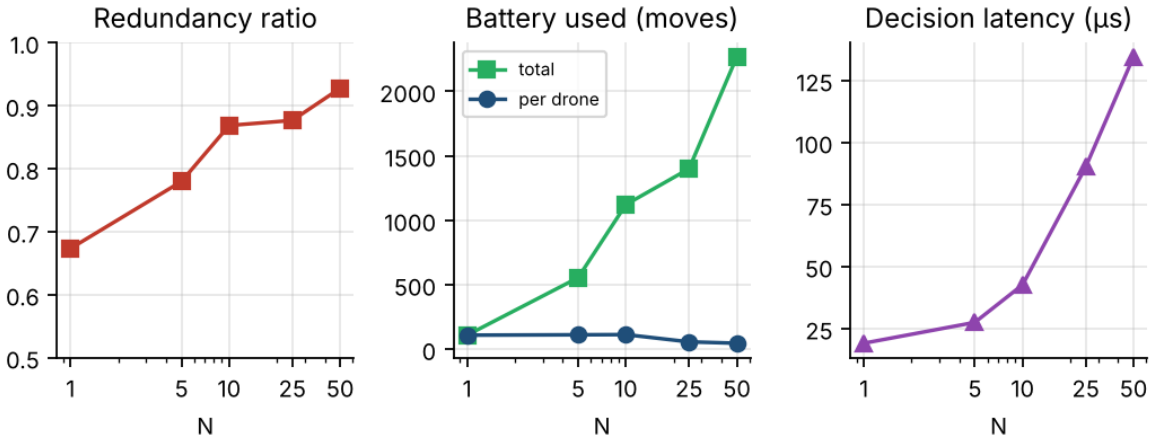


Figure B1c – Redundancy, battery and decision latency against N (no failures).

Case (N = 10, 30 seeds)	success	final cov.	msgs/tick (all)	max msgs at one node/tick
swarm | links ok	57%	93%	30	9
central | links ok	60%	95%	8	10
central | 30% link loss	67%	95%	9	10
central | controller dies t=60	23%	84%	11	10
swarm | 30% drone failure	50%	92%	28	9
central | 30% drone failure	53%	93%	9	10

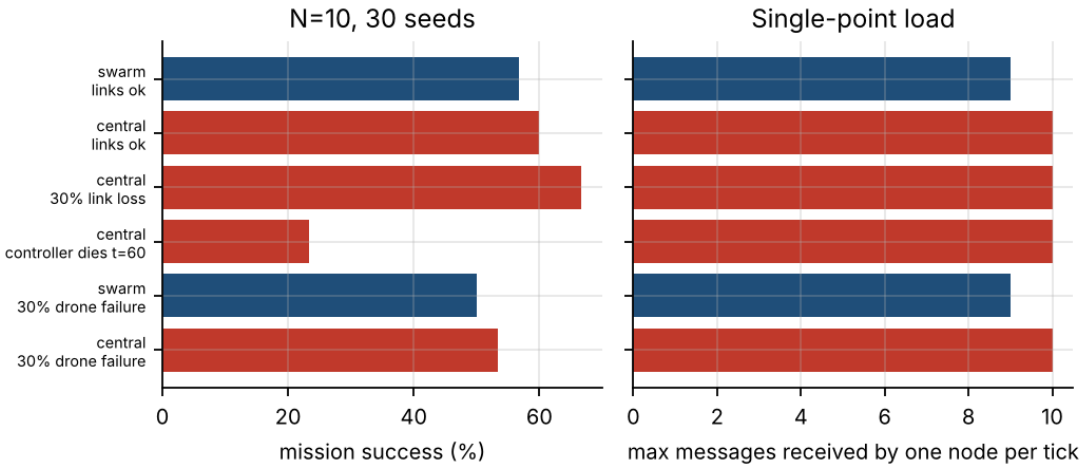


Figure B1d – Central controller versus swarm (N = 10).

Interpretation of scaling behaviour.

- **One drone cannot do the mission** (0% success, ≈20% coverage): it simply runs out of battery. **N = 5** raises coverage to ≈74% but still only 7% success. **N = 10** is the knee (57% success, 93% coverage). **N = 25 and 50** reach 100% success and 100% coverage in every run, with time to 90% falling from 49 to 40 ticks.
- **Diminishing returns.** Going from 25 to 50 drones saves only ≈9 ticks but costs 868 more battery moves in total, raises redundancy from 0.88 to 0.93, multiplies decision latency by 1.5× (map merging with up to 49 neighbours on the crowded launch pad) and radio traffic grows roughly with N × neighbours. Past N ≈ 25 the swarm is limited by the launch pad and travel distance, not by drone count.
- **Robustness.** With 10% and 30% of drones failing, N = 25 and 50 still succeed in 100% of runs (the redundant drones absorb the loss), while N = 10 drops from 57% to 50% success. The 10%-failure N = 10 figure (63%) is higher than the 0% figure only because of random variation over 30 seeds (about ±9 percentage points).
- **Safety finding.** At N = 5 and N = 10 some drones ran out of battery before reaching home (column ‘drones out of battery’: on average 0.6 of 5 and 1.7 of 10 drones with no failures). The fixed reserve of 6 units does not cover detours around buildings. The adaptive drain estimate and a path-length-based (not Manhattan) return estimate from B1.1 should be added to the swarm drones.
- **Central versus swarm.** With perfect links the central controller is slightly better (60% vs 57%) because it has global knowledge: decentralisation is not free. But if the controller fails the central design collapses to 23% while the swarm is unaffected, and its single node must receive 10 messages per tick (50 at N = 50). This confirms the diagnosis of A2.1.
- **Limits of the evidence.** One map size, one battery size, synthetic survivors; 30 seeds give about ±9 pp uncertainty on a success rate near 50%.

TASK B2 – Engineer an Adversarial Input for the Expert System (5 marks)

B2.1 – Implement and evaluate (2 marks)

Source code: expert_shell.py (engine, parser, HOW/WHY, run-time rule add/remove: no pump knowledge inside) and b2_pump.py (rule base and cases). 20 rules, 5 fault types; rules were written using only the 15 training cases (3 per fault); 15 cases (3 per fault) were kept as hold-out.

Rule
R01 | IF flow=low AND discharge_pressure=low THEN hydraulic_loss=yes | PRIORITY 1
R02 | IF noise=grinding AND vibration=high THEN mechanical_damage=yes | PRIORITY 1
R03 | IF bearing_temp=high THEN heat_source=bearing | PRIORITY 1

R04 | IF motor_temp=high THEN heat_source=motor | PRIORITY 1
R05 | IF noise=crackling AND suction_pressure=low AND vibration=high THEN fault=cavitation | PRIORITY 4
R06 | IF noise=crackling AND suction_pressure=low THEN fault=cavitation | PRIORITY 4
R07 | IF suction_pressure=low AND flow=low AND vibration=high THEN fault=cavitation | PRIORITY 4
R08 | IF leak_visible=yes THEN fault=seal_leak | PRIORITY 3
R09 | IF leak_visible=yes AND discharge_pressure=low AND vibration=normal THEN fault=seal_leak | PRIORITY 3
R10 | IF mechanical_damage=yes AND heat_source=bearing THEN fault=bearing_failure | PRIORITY 5
R11 | IF noise=grinding AND vibration=high THEN fault=bearing_failure | PRIORITY 5
R12 | IF bearing_temp=high AND vibration=high THEN fault=bearing_failure | PRIORITY 5
R19 | IF discharge_pressure=low AND flow=normal AND vibration=normal AND leak_visible=no THEN fault=seal_leak | PRIORITY 3
R20 | IF noise=grinding AND bearing_temp=high THEN fault=bearing_failure | PRIORITY 5
R13 | IF hydraulic_loss=yes AND motor_current=low THEN fault=impeller_wear | PRIORITY 2
R14 | IF flow=low AND motor_current=low AND suction_pressure=normal THEN fault=impeller_wear | PRIORITY 2
R15 | IF hydraulic_loss=yes AND vibration=high AND noise=none THEN fault=impeller_wear | PRIORITY 2
R16 | IF motor_current=high AND heat_source=motor THEN fault=motor_overload | PRIORITY 5
R17 | IF flow=zero AND discharge_pressure=high THEN fault=motor_overload | PRIORITY 5
R18 | IF motor_current=high AND discharge_pressure=high AND flow=low THEN fault=motor_overload | PRIORITY 5

Strategy	Train accuracy (15)	Hold-out accuracy (15)	Rules fired / case
specificity	73%	67%	2.47
priority	73%	67%	2.47
recency	80%	73%	2.47
order	73%	67%	2.47

Fault (specificity strategy, hold-out)	precision	recall	support
cavitation	1.00	1.00	3
seal_leak	0.75	1.00	3
bearing_failure	1.00	1.00	3
impeller_wear	n/a (no prediction)	0.00	3
motor_overload	1.00	0.33	3

Case	True fault	Diagnosis	Rules fired (in order)
4	cavitation	cavitation	R05, R06
5	cavitation	cavitation	R05, R07, R06
6	cavitation	cavitation	R05, R07, R01, R06, R04
10	seal_leak	seal_leak	R09, R08
11	seal_leak	seal_leak	R09, R08
12	seal_leak	seal_leak	R09, R08
16	bearing_failure	bearing_failure	R02, R11, R12, R20, R03, R10
17	bearing_failure	bearing_failure	R12, R03
18	bearing_failure	bearing_failure	R20, R03
22	impeller_wear	undetermined	none
23	impeller_wear	seal_leak	R09, R14, R01, R13, R08
24	impeller_wear	undetermined	R01
28	motor_overload	undetermined	none
29	motor_overload	motor_overload	R18, R04, R16
30	motor_overload	undetermined	R01, R04

Sample explanations (HOW):

```
Case 16 (bearing failure):  fault=bearing_failure because R11 (noise=grinding
[given]; vibration=high [given]).
Case 6  (cavitation):       fault=cavitation because R05 (noise=crackling
[given]; suction_pressure=low [given]; vibration=high [given]).
WHY (asked during case A2 below): I need 'suction_pressure' because it is a
premise of: R05->fault=cavitation, R06->fault=cavitation, R07->fault=cavitation,
R14->fault=impeller_wear
```

Interpretation. Hold-out accuracy is 67% (specificity) against 73% on the training cases; the errors are cases whose sensor readings were corrupted (e.g. flow = zero for an impeller fault, no leak/current evidence), for which no rule exists, so the system answers 'undetermined' rather than guessing. Recall is lowest for impeller_wear and motor_overload. Only 15 test cases are used, so one case is worth 6.7 percentage points: do not read the small differences between strategies as significant.

B2.2 – Engineer adversarial cases (2 marks)

Each prediction below was written into the program before it was run (see b2_run.py: the ADV list holds the predictions).

Adversarial case	Prediction (specificity / priority / recency / order)	Actual	Match
A1 conflicting rules (bearing vs cavitation evidence) (true: unknown)	cavitation / bearing_failure / cavitation / cavitation	cavitation / bearing_failure / cavitation / cavitation	all 4
A2 missing evidence (only noise=crackling known) (true: cavitation)	undet. / undet. / undet. / undet.	undet. / undet. / undet. / undet.	all 4
A3 equal-specificity tie (seal R09 vs impeller R14) (true: impeller_wear)	seal_leak / seal_leak / seal_leak / seal_leak	seal_leak / seal_leak / impeller_wear / seal_leak	3 of 4
A4 contradictory sensors (flow=zero but current low) (true: unknown)	undet. / undet. / undet. / undet.	undet. / undet. / undet. / undet.	all 4
A5 redundant evidence (full bearing picture) (true: bearing_failure)	bearing_failure / bearing_failure / bearing_failure / bearing_failure	bearing_failure / bearing_failure / bearing_failure / bearing_failure	all 4

What went wrong with my prediction. 19 of 20 predictions were right. For A3 under recency I predicted seal_leak but the shell answered impeller_wear: I forgot that rule R13 (impeller) uses the derived fact hydraulic_loss, which is newer than any raw sensor fact, so recency prefers it. A prediction error that exposes how recency interacts with chained rules.

Missing evidence (A2). With only noise = crackling, no rule can fire, so the answer is 'undetermined' for all strategies. When the shell is allowed to ask (WHY facility) it asks one question – suction_pressure – and then diagnoses cavitation.

Conflicting rules / rule order (A1, A3). In A1 the correct answer is unknown but the strategies disagree (priority says bearing_failure, others cavitation): the conflict-resolution strategy is part of the knowledge. In A3 (true fault: impeller wear) every strategy was wrong in the original order: R09 and R14 both have 3 premises and R09 comes first; priority prefers seal (3) over impeller (2) always.

Strategy	Hold-out accuracy	Same answer after random re-ordering of the rules (mean over 15 hold-out cases, 100 shuffles)	Cases that are order-sensitive	A3 stability (200 shuffles)

specificity	67%	96.9%	1	58%
priority	67%	100.0%	0	100%
recency	73%	96.9%	1	56%
order	67%	97.3%	1	56%

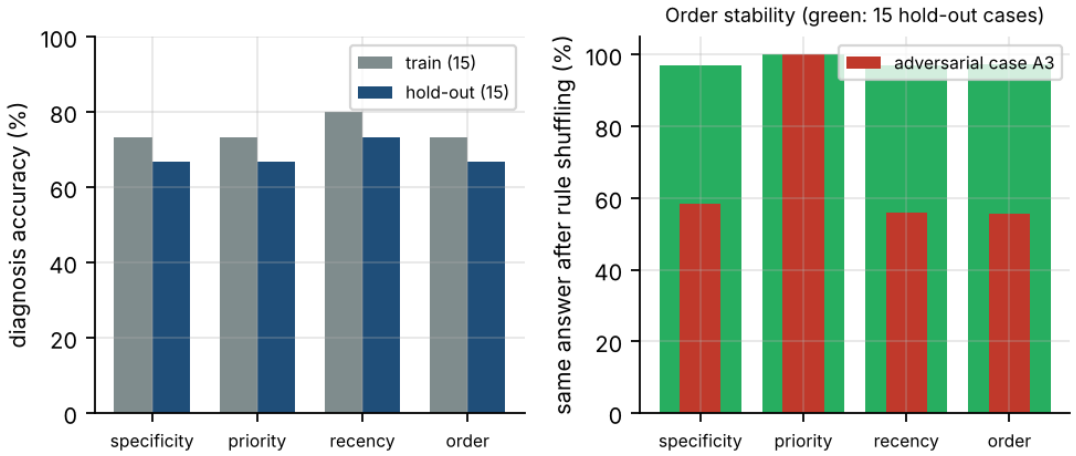


Figure B2 – Strategy comparison and order stability.

Strategy comparison. Priority is perfectly order-stable (100%) but its answers are fixed by severity labels, not by evidence (A3: always wrong). Specificity uses the evidence (more premises = stronger case) but ties are broken by rule order, so A3 gives a different answer in about 41% of shuffles. Recency behaves like specificity with extra chaining effects. Order (first match) is the least principled and the most order-dependent. Recommended: specificity with an explicit priority tie-break, never rule order alone.

B2.3 – Knowledge-acquisition test (1 mark)

Engine code untouched; a rule is added by text only:

```
R21 | IF noise=humming AND flow=low THEN fault=motor_overload | PRIORITY 5
```

Case	True fault	Before	After	Effect
28	motor_overload	undetermined	motor_overload	fixed
30	motor_overload	undetermined	motor_overload	fixed

Regression run over all 30 cases: 2 diagnoses changed, all improvements, none broken (hold-out accuracy 67% → 80%; train 73% → 73%). Second edit – tightening R09 by adding premise flow = normal (remove old R09, add new text) together with R21 – fixed cases 19, 23, 28, 30; hold-out accuracy becomes 87%. **Caution:** these edits were designed after seeing hold-out errors, so the hold-out set is no longer an unbiased estimate; for a fair figure collect new cases.

TASK B3 – Applied Case: Full Model versus Edge Model (8 marks)

Case: (a) AI-based intrusion detection. **Dataset:** NSL-KDD (Tavallaee, Bagheri, Lu, Ghorbani, 2009), KDDTrain+ and KDDTest+, mirror github.com/defcom17/NSL_KDD. Binary target: normal vs attack. **Code:** intrusion.py, spark_scale.py.

B3.1 – Correctness on a small benchmark (2 marks)

Split and leakage control. NSL-KDD has no timestamp, so a time-ordered split is impossible; the official source-wise split is used: model, one-hot encoder, scaler and feature ranking are fitted on KDDTrain+ (125,973 records, 46.5% attacks) only and evaluated on KDDTest+ (22,544 records, 56.9% attacks).

KDDTest+ contains 17 attack types never seen in training (3,750 of 12,833 test attacks), which is a deliberate distribution shift. **Full model:** random forest, 200 trees, no depth limit, seed 0 (fitted in 14 s). Baseline: logistic regression.

Model (KDDTest+)	accuracy	precision	recall	F1	ROC-AUC	PR-AUC
Full random forest	76.9%	96.8%	61.5%	75.2%	0.961	0.964
Logistic regression (all features)	75.3%	91.7%	62.3%	74.2%	0.792	0.865

Confusion matrix (full)	predicted normal	predicted attack
actual normal	9,450	261
actual attack	4,937	7,896

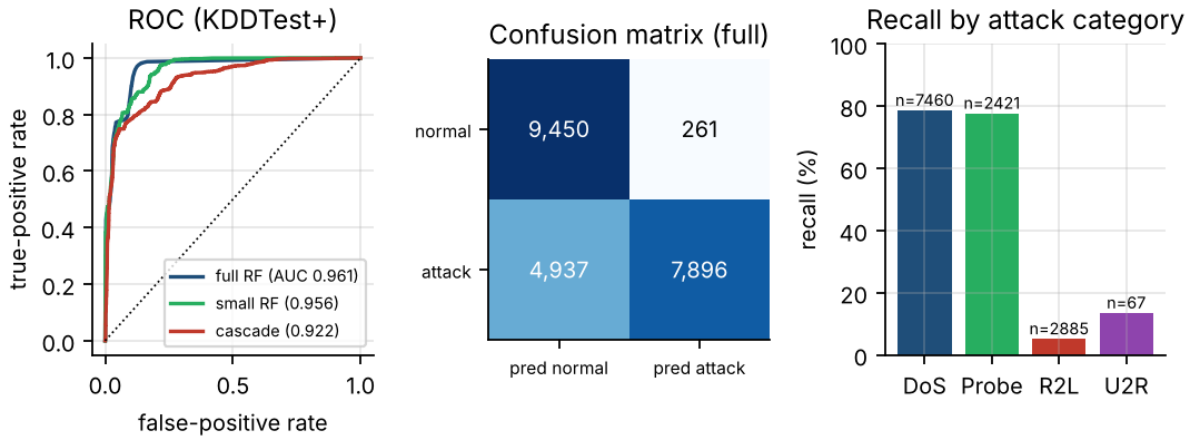


Figure B3a – ROC curves, confusion matrix and per-category recall.

Interpretation. The ranking quality is high (ROC-AUC 0.961) and precision is 96.8%: when it raises an alert it is almost always right. But recall is only 61.5% (accuracy 76.9%), far below the 99% often quoted for the old KDD'99 set, because KDDTest+ is hard: it is a different distribution from the training data. The logistic baseline has similar accuracy but a much worse AUC (0.792), so the forest ranks traffic better. Accuracy alone would hide that missing 38% of attacks is the real problem.

B3.2 – Find a counterexample (2 marks)

Error category	Count in KDDTest+	Example (sampled, seed 0)
FN unseen attack type	2,832	mscan on telnet/REJ, src_bytes=0, count=1, p_attack=0.32; mscan on imap4/RSTO, src_bytes=0, count=2, p_attack=0.345
FN seen type, R2L/U2R (low-volume)	2,089	guess_passwd on telnet/SF, src_bytes=123, count=1, p_attack=0.07; warezmaster on ftp/SF, src_bytes=160, count=1, p_attack=0.39
FN seen type, DoS/Probe	16	back on http/SF, src_bytes=54540, count=3, p_attack=0.435; back on http/SF, src_bytes=54540, count=2, p_attack=0.425
FP normal traffic	261	normal on private/SF, src_bytes=1, count=134, p_attack=0.985; normal on other/RSTR, src_bytes=0, count=1, p_attack=0.96

Eight sampled mis-predictions in four categories (counts in the table above). (1) Unseen attack types (mscan, apache2, snmpguess, processtable …) look like ordinary traffic to a model that never saw them. (2) Low-volume R2L/U2R attacks (guess_passwd, warezmaster) are single connections with normal-looking flags and small byte counts. (3) Seen DoS 'back' attacks that carry large payloads (DoS/Probe types the model normally catches). (4) False alarms on normal traffic with high connection counts or reset flags.

Attack class	n in test	recall
DoS	7460	78.6%
Probe	2421	77.6%
R2L	2885	5.1%
U2R	67	13.4%
ALL types seen in training	9083	76.8%
ALL types NOT seen in training	3750	24.5%

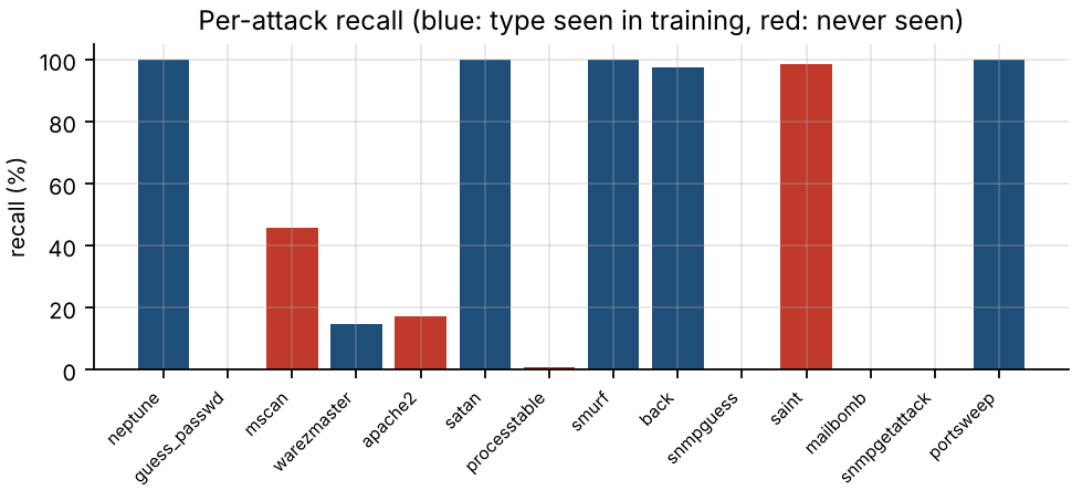


Figure B3b – Recall per attack type: the model fails on unseen types and on password-guessing R2L attacks.

Failure class demonstrated: attack types not present in training are detected only 24% of the time versus 77% for seen types, and R2L attacks (remote password guessing, warez, mail abuse) only 5%. For guess_passwd, a seen type with 1,231 test records, recall is 0%. This is a limit of supervised signature-style learning, not a bug.

B3.3 – Increase the problem size with the Task A4 pipeline in PySpark (2 marks)

Pipeline: exactly the pipeline of Task A4 in spark_scale.py: records (service code, record id, log1p(src_bytes) from NSL-KDD) generated inside the executors; map → map → aggregateByKey (per-service count, mean, max, std) → takeOrdered(10); a third action then re-reads the raw data to count readings above mean + 3σ (this is what makes caching the raw RDD worthwhile). Spark 4.3.0, local mode on 1 CPU core, 1.5 GB driver memory. Times are wall-clock for the three actions together.

records	partitions	no cache (s)	cache raw RDD (s)	cache stats (s)	shuffle stages executed (all 3 actions)
100,000	1	2.92	0.79	0.69	1
100,000	2	0.88	0.84	0.79	1
100,000	4	1.39	1.54	1.35	1
100,000	8	2.55	2.34	2.44	1
1,000,000	1	2.27	4.64	2.27	1
1,000,000	2	2.35	2.57	2.57	1
1,000,000	4	2.80	2.88	2.96	1
1,000,000	8	3.43	3.82	3.31	1
10,000,000	1	17.87	16.64	not run	1
10,000,000	2	17.81	16.51	not run	1
10,000,000	4	19.08	16.99	not run	1
10,000,000	8	20.36	18.90	not run	1

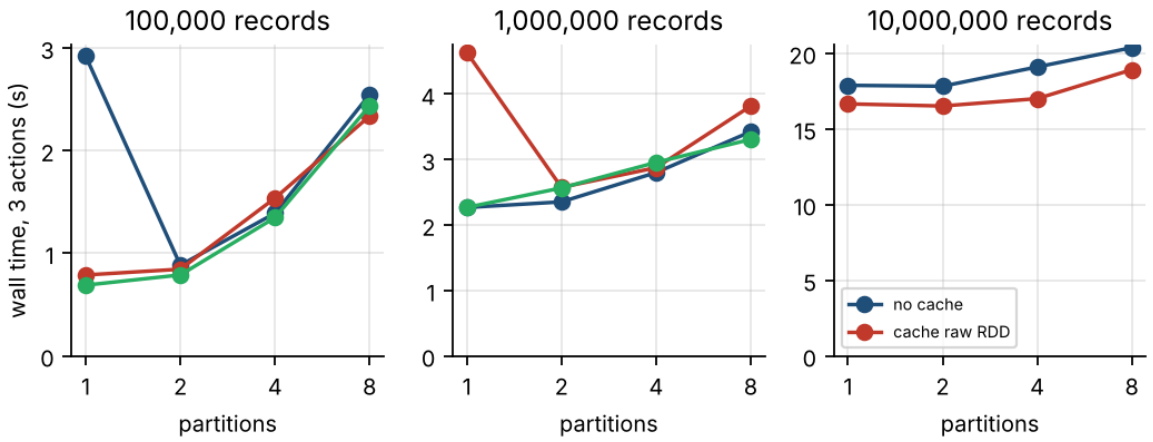


Figure B3c – Runtime against partitions, with and without caching.

Interpretation. (1) Scaling with data: 10× more records cost about 6.8× more time (2.8 s at 10^6, 19 s at 10^7, partitions = 4; the fixed start-up cost makes the small runs look relatively slow), i.e. roughly linear, throughput about 524k records/s. (2) More partitions did not help; they hurt slightly: at 10^7 the time rises from 17.9 s (1 partition) to 20.4 s (8 partitions). The sandbox has **1 CPU core**, so extra partitions cannot run in parallel; they only add task scheduling, Python-worker start-up and more, smaller shuffle blocks (8 × 70 map outputs instead of 1 × 70). With 4 real cores the same experiment is expected to speed up until partitions ≈ cores; this was not measurable here. (3) Shuffles: the pipeline has exactly one wide dependency (aggregateByKey); lineage shows 1 ShuffledRDD, and Spark executed that shuffle stage once per run, not once per action, because it re-uses shuffle files (the stage is skipped in later actions). So caching stats brought no measurable gain (differences of up to ±0.3 s are noise). (4) Caching raw RDD: the first action is slower (blocks must be built and stored: 12.7 s versus 9.7 s at 10^7), but the third action, which re-reads the data, is 2.3× faster (4.0 s vs 9.1 s). Net gain is only about 7–11% at 10^7 for 96 MB of RAM, and at 10^6 caching is a net loss. Caching the raw data pays off only when it is re-read several times and fits in memory.

B3.4 – Engineering trade-off: implement the Task A5 compressed model (2 marks)

Models: full RF (200 trees); small forest = top-20 features, 25 trees, depth ≤ 12, min leaf 5; cascade = logistic regression on the same 20 features with early exit if p ≤ 0.05 or ≥ 0.95, else the small forest. Latency = single-sample prediction call, 1,500 samples, this CPU; throughput = batch of 20,000. Edge projection = ×8 (assumption).

Model	latency mean / p95 (ms) here	projected edge (×8)	throughput (samples/s, batch)	size	accuracy	recall	F1	AUC	meets 50 ms (p95, edge)?
Full RF	9.72 / 14.31	77.7 / 114.5	89,086	18.83 MB	76.9%	61.5%	75.2%	0.961	NO
Small forest	1.38 / 1.97	11.1 / 15.8	879,024	1.00 MB	76.3%	60.6%	74.4%	0.956	yes
Cascade	0.59 / 1.99	4.7 / 15.9	38,892,422	1.00 MB	74.9%	58.1%	72.5%	0.922	yes

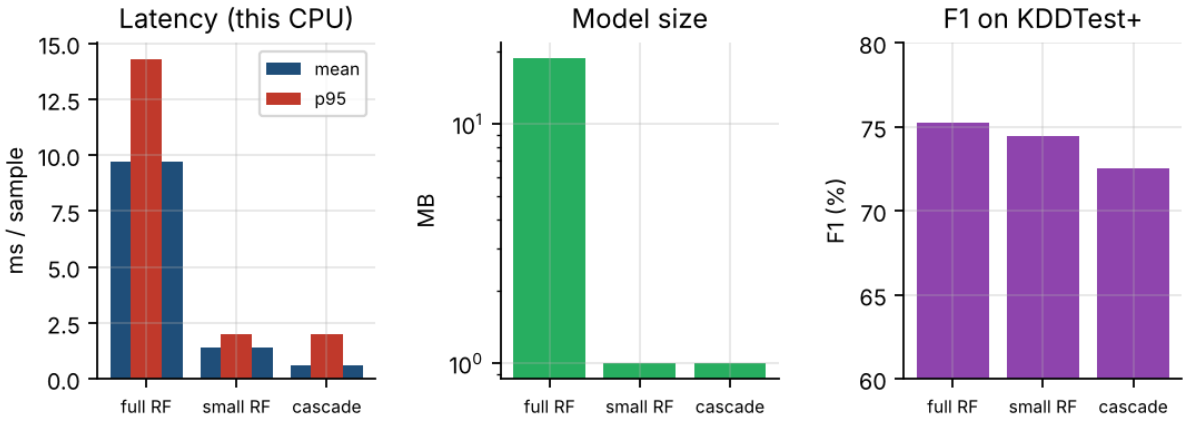


Figure B3d – Latency, size and F1 of full and compressed models.

Accuracy change. Small forest: accuracy −0.6 points, recall −1.0 points, F1 −0.8 points, AUC −0.005, for 19× smaller size and 7.0× lower latency. Cascade: 70% of samples exit at stage 1 (mean latency 0.59 ms) but accuracy falls 2.0 points, recall 3.4 points and AUC to 0.922, because 14% of early exits are wrong (the linear stage is confidently wrong on unseen attacks). Its p95 equals the small forest's, because the 30% uncertain samples still run the forest.

Is the 50 ms budget met? On this CPU, yes for all three (full: p95 14.3 ms). On the assumed 8×-slower edge device: **full model no** (p95 ≈ 114 ms, size 18.8 MB), **small forest yes** (p95 ≈ 16 ms, 1.0 MB), cascade yes. Recommendation: deploy the **small forest**; the cascade adds speed that is not needed and costs 1.4 more accuracy points. **Caveat:** the ×8 factor is an assumption; the claim is only valid after measuring on the target device. Also: single-sample sklearn calls carry Python overhead; an exported C/ONNX tree evaluator would be faster.

TASK B4 – Final Engineering Decision: Conversational Interface and Deployment (5 marks)

B4.1 – Implement a practical strategy (2 marks)

Code: chatbot.py. Retrieval chatbot over the Task B3 system with 9 intents (list_open_alerts, alert_count, why_flagged, top_attack_type, model_accuracy, latency_status, acknowledge_alert, retrain_model, greeting_help) plus an out-of-scope fallback. Features: TF-IDF over character 3–5-grams; one centroid per intent; prediction = most similar centroid by cosine similarity, or 'out_of_scope' if the cosine is below the threshold θ. Data (hand-written by me): 8 training, 3 development and 5 test utterances per intent; 12 OOS development and 30 OOS test utterances. Design choices (n-gram type, centroid vs nearest-neighbour) and θ = **0.33** were chosen on the development set only; the test set was used once. A first word-bigram variant scored 56% on the development set and was discarded.

Intent	precision	recall	F1	support
list_open_alerts	0.80	0.80	0.80	5
alert_count	1.00	0.60	0.75	5
why_flagged	1.00	0.40	0.57	5
top_attack_type	1.00	1.00	1.00	5
model_accuracy	0.56	1.00	0.71	5
latency_status	0.00	0.00	0.00	5
acknowledge_alert	1.00	0.80	0.89	5
retrain_model	1.00	1.00	1.00	5
greeting_help	0.67	0.40	0.50	5
out_of_scope	0.67	0.87	0.75	30

In-scope intent accuracy **67%** (45 test utterances), out-of-scope rejection **87%**, false-accept rate **13%** (30 OOS utterances), overall accuracy 75%. Response latency (intent decision only): **0.29 ms mean, 0.34 ms p95**. Test sets are small (5 per intent, so one utterance = 20 points of recall).

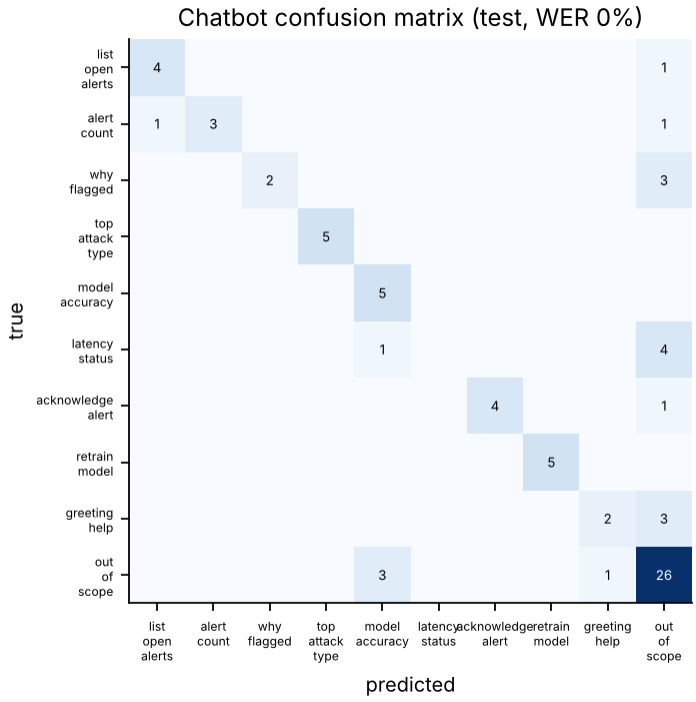


Figure B4a – Confusion matrix on the test set (last row/column: out_of_scope).

Errors and interpretation. Most mistakes are in-scope questions rejected as out-of-scope (e.g. “are predictions fast enough”, “explain the decision behind this alert”): their wording shares few character n-grams with the 8 training examples of the intent. latency_status has F1 = 0.00: none of its five paraphrases (“delay”, “speed of inference”, “time budget”) reached the threshold, and one went to model_accuracy. Remedies: more and more varied training utterances per intent, synonym expansion, or an embedding model; lowering θ would admit more out-of-scope questions (see the sweep).

Sample dialogue (live answers from the Task B3 scored test set):

```
User: Which alerts are open?
Bot : [list_open_alerts, cos=0.71] 8157 alerts are open; highest-risk: neptune
on service supdup (p=1.00).
User: Why was this flagged?
Bot : [why_flagged, cos=0.57] Top evidence: src_bytes, dst_bytes, same_srv_rate,
diff_srv_rate, flag (from the random-forest importances).
User: what's the weather like?
Bot : [out_of_scope, cos=0.23] Sorry, I can only help with the detection system.
```

B4.2 – Challenge your strategy: simulated speech-recognition errors (1 mark)

Each test utterance is corrupted word by word with probability WER by a random substitution, deletion or insertion (vocabulary taken from the training utterances); 20 corruption seeds per WER; θ fixed at the development-selected value.

WER	in-scope intent accuracy (mean ± sd)	out-of-scope detection	false-accept rate	in-scope rejected as OOS
0%	66.7% ± 0.0	86.7%	13.3%	28.9%

10%	59.1% ± 4.4	89.2%	10.8%	35.3%
20%	48.2% ± 5.3	89.7%	10.3%	46.1%
30%	42.1% ± 3.9	87.5%	12.5%	52.3%

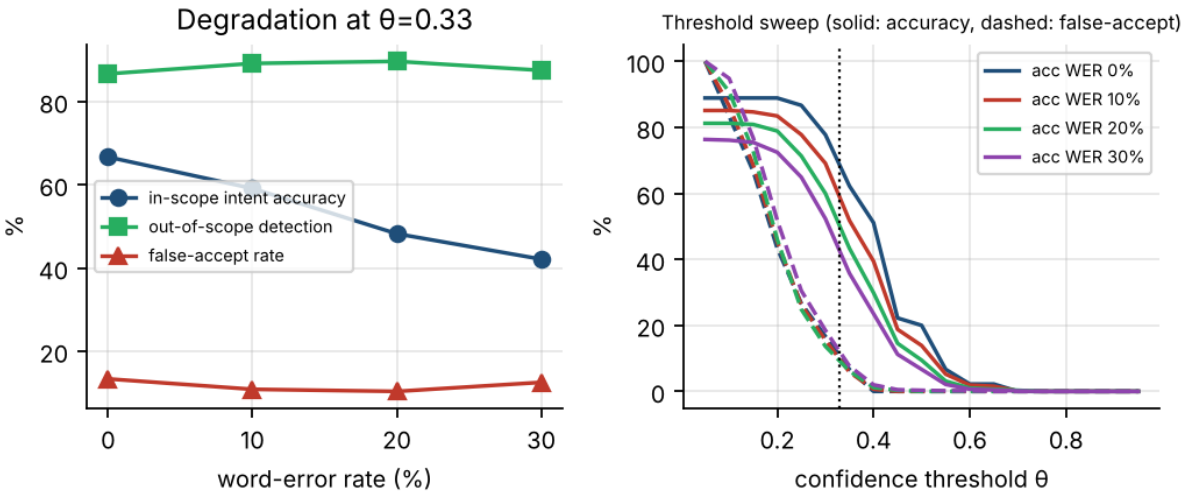


Figure B4b – Left: degradation with WER. Right: threshold sweep (solid: in-scope accuracy, dashed: false-accept rate).

Interpretation. Intent accuracy falls from 67% to 42% as WER rises to 30%, mostly because corrupted in-scope questions fall below θ and are rejected (false-reject 29% → 52%); the false-accept rate stays between 10 and 13% (it does not improve with noise: some random substitutions make OOS sentences look similar to an intent). The sweep shows the trade-off: raising θ reduces false accepts but sacrifices accuracy; lowering θ does the opposite. With ASR errors the usable operating point shifts to a lower θ, but then a security console risks acting on a wrong command: for **acknowledge_alert** (state-changing) a confirmation step ("Do you want to close alert 17?") is required.

B4.3 – Production decision (2 marks)

Criterion	Evidence	Favours
Latency	Detector: small forest p95 2.0 ms here (≈16 ms projected edge) versus full 14.3 ms (≈114 ms projected). A cloud call adds a network round trip (assumed 40–100 ms, not measured) to every sample, so it alone can break the 50 ms budget. Chatbot: 0.29 ms.	Detector on edge; chatbot either
Accuracy	Full model F1 75.2% / AUC 0.961 versus small forest 74.4% / 0.956: the loss from compression is 0.8 F1 points, small compared with the unseen-attack gap (recall 24% on unseen types).	Edge acceptable
Throughput	Small forest batch throughput 879,024 samples/s here (≈109,878/s projected), single-sample ≈ 90/s per core on the edge. Cloud scales horizontally but needs the network to carry every flow.	Edge unless link rate > capacity
Privacy	Connection features describe internal hosts and services; keeping scoring on the device means only alerts leave the site.	Edge
Cost (assumption)	Edge: one-off hardware, model 1.0 MB over the air. Cloud: pay per call plus bandwidth for all traffic; no price data was measured.	Edge for high volume (assumed)
Chatbot / voice	Needs the alert database, operator authentication and (for voice) heavy speech recognition; not latency-critical (operators wait seconds); intent model is tiny and cheap either way.	Operator-side server / cloud

Decision. Run the **compressed detection model (small forest) on the edge device**, and run the **assistant**

Appendix – files and how to rerun

File	Purpose
drone_agent.py	Single-drone simulator and layered agent; scenarios S1–S5 and A1.5 (B1.1, B1.2)
swarm.py	Swarm and central-controller simulator, 30-seed experiments (B1.3, B1.4)
expert_shell.py, b2_pump.py, b2_run.py	Expert-system shell, pump rule base and cases, adversarial and regression tests (B2)
intrusion.py	NSL-KDD full and compressed models, metrics, error analysis (B3.1, B3.2, B3.4)
spark_scale.py	PySpark RDD pipeline for 10^5–10^7 records (B3.3)
chatbot.py	Retrieval chatbot, WER stress test, threshold sweep (B4)
a2_trace.py, figs.py, build_doc.py	Swarm trace check, figures, this document